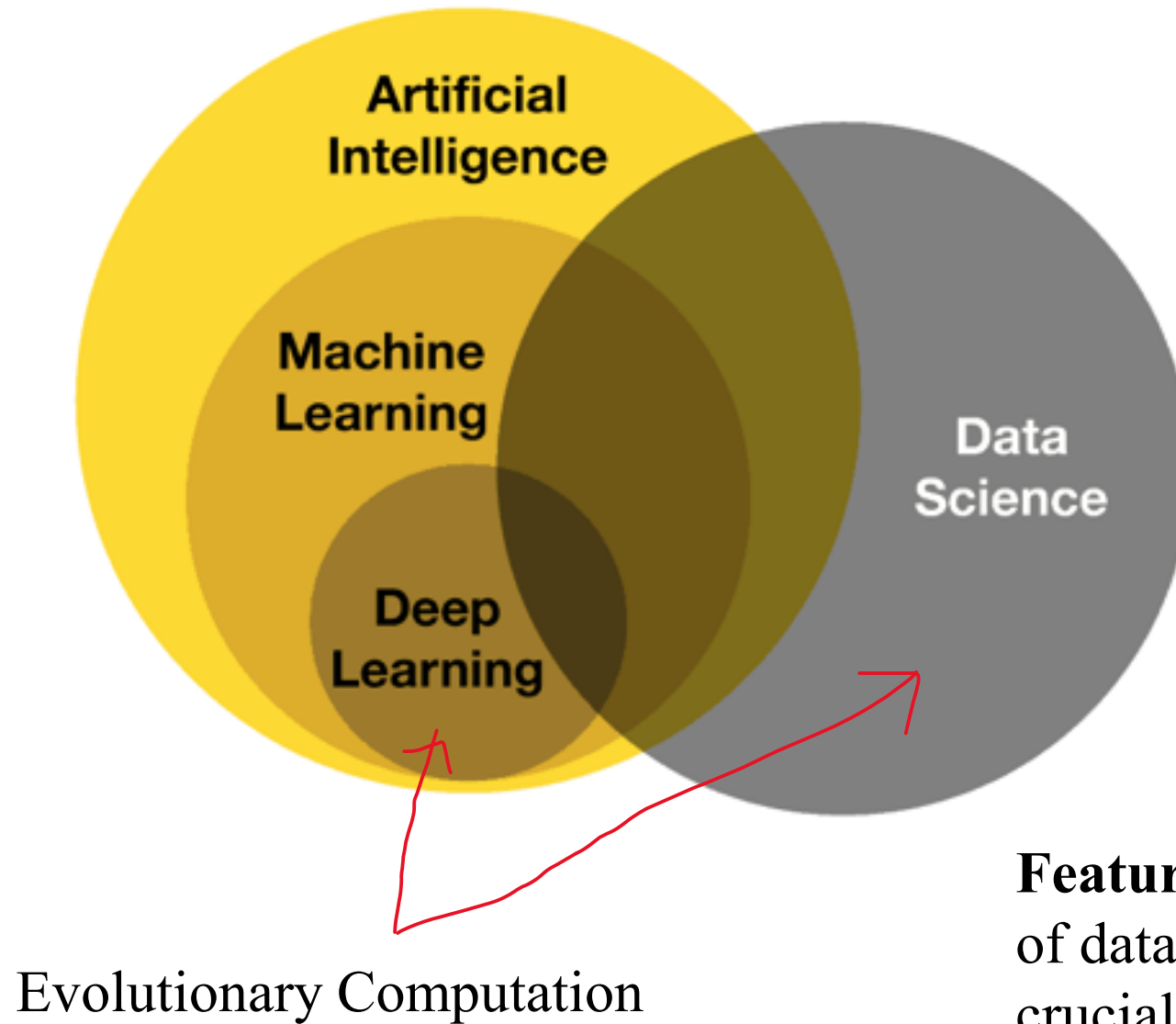
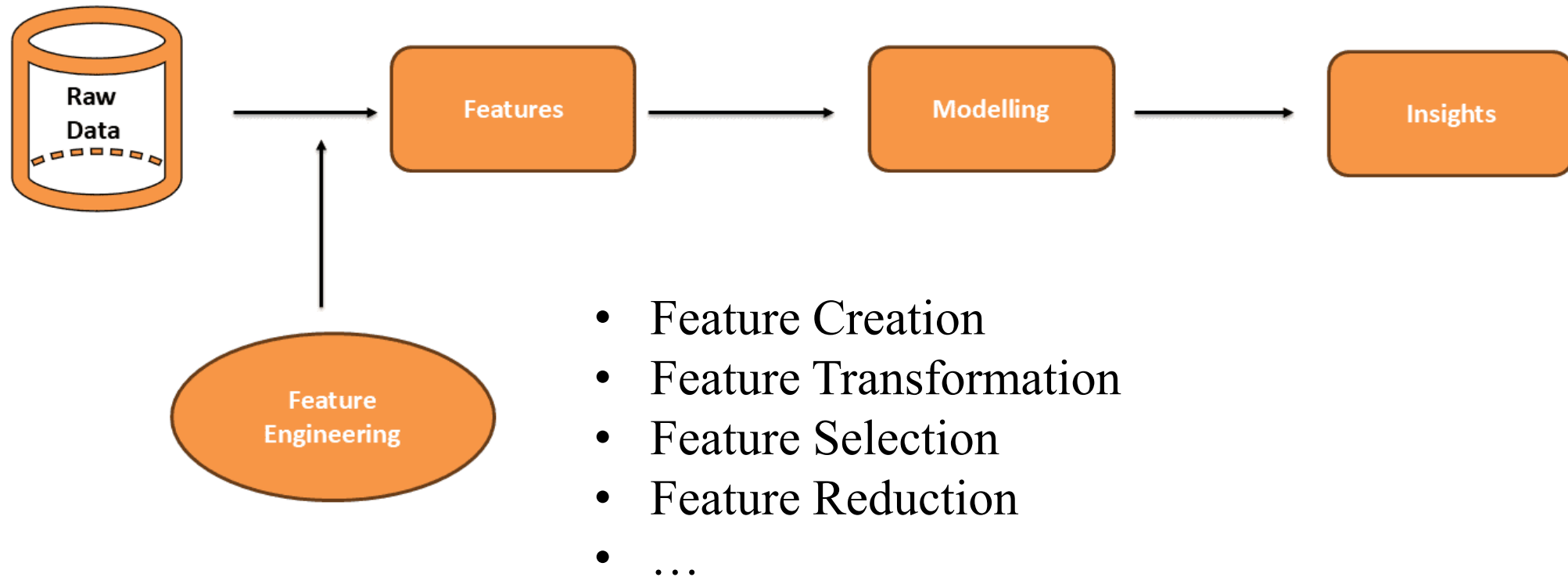


Introduction to Feature Engineering and EC

CJ Chung

Updated Dec 3, 2025





<https://machinelearningmastery.com/the-concise-guide-to-feature-engineering-for-better-model-performance>

Some FE Techniques in Python

Why is Feature Engineering Important?

- **Improved Accuracy:** Good features help the model learn better patterns. This leads to more accurate predictions.
- **Reduced Overfitting:** Better features help the model generalize well to new data. This reduces the chance of overfitting.
- **Algorithm Flexibility:** Many algorithms work better with clean and well-prepared features.
- **Easy Interpretability:** Clear features make it easier to understand how the model makes decisions.
- **Reducing Computational Complexity:** Techniques like PCA or manual feature selection eliminate redundant and irrelevant features, making ML/DL models faster to train.

Feature Engineering (FE) Tasks

- *New Feature Creation*
- Feature Transformation
- Feature Selection
- Feature Reduction (Dimensionality Reduction)
- Others Techniques
 - Handling Missing Values
 - Feature Encoding for Complex Data
 - Feature Noise Reduction
 - Sparse Data Handling
 - Embedding Features:

Feature Creation (1 /2)

- **Polynomial Features:** Create higher-order terms (e.g., x^2 , x^3) - Polynomial features involve creating new features by raising existing ones to a power (e.g., square, cube) or by combining them in polynomial combinations. These transformations allow models to capture non-linear relationships in the data, which may not be evident using only the original features.
- **Aggregation:** Summarize features, such as mean, median, or sum (e.g., for time series or grouped data).
- **Domain-Specific Features:** Generate features using domain knowledge (e.g., BMI = weight/height²).
- **Date/Time Features:** Extract parts of dates or timestamps (e.g., day, month, day of the week, hour).

Feature Creation (2 /2)

- Text Features: Extract keywords, word counts, or embeddings (dense, low-dimensional vector representations of words) from textual data.
- Feature Interaction: Combine two or more features (e.g., $x_1 + x_2$ or $x_1 \times x_2$).
- Feature Crossing: the process of creating new features by combining two or more existing features in a dataset. This is often done by multiplying, *concatenating*, or applying other transformations to the original features to capture interactions between them.
Concatenating example: Combining "City" and "State" to create a "City-State" feature.

Feature Transformation

- **Scaling and Normalization:**
 - Standardization: Rescale to have zero mean and unit variance.
 - Min-Max Scaling: Scale values to a fixed range, like $[0, 1]$.
- **Log Transformation:** Handle skewed distributions by applying logarithms.
- **Encoding Categorical Features:**
 - One-hot encoding, label encoding, or target encoding.
- **Binning:** Group continuous values into discrete intervals (e.g., low, medium, high).
- **Box-Cox or Yeo-Johnson Transformations:** Make non-normal data more normal-like.

Feature Selection

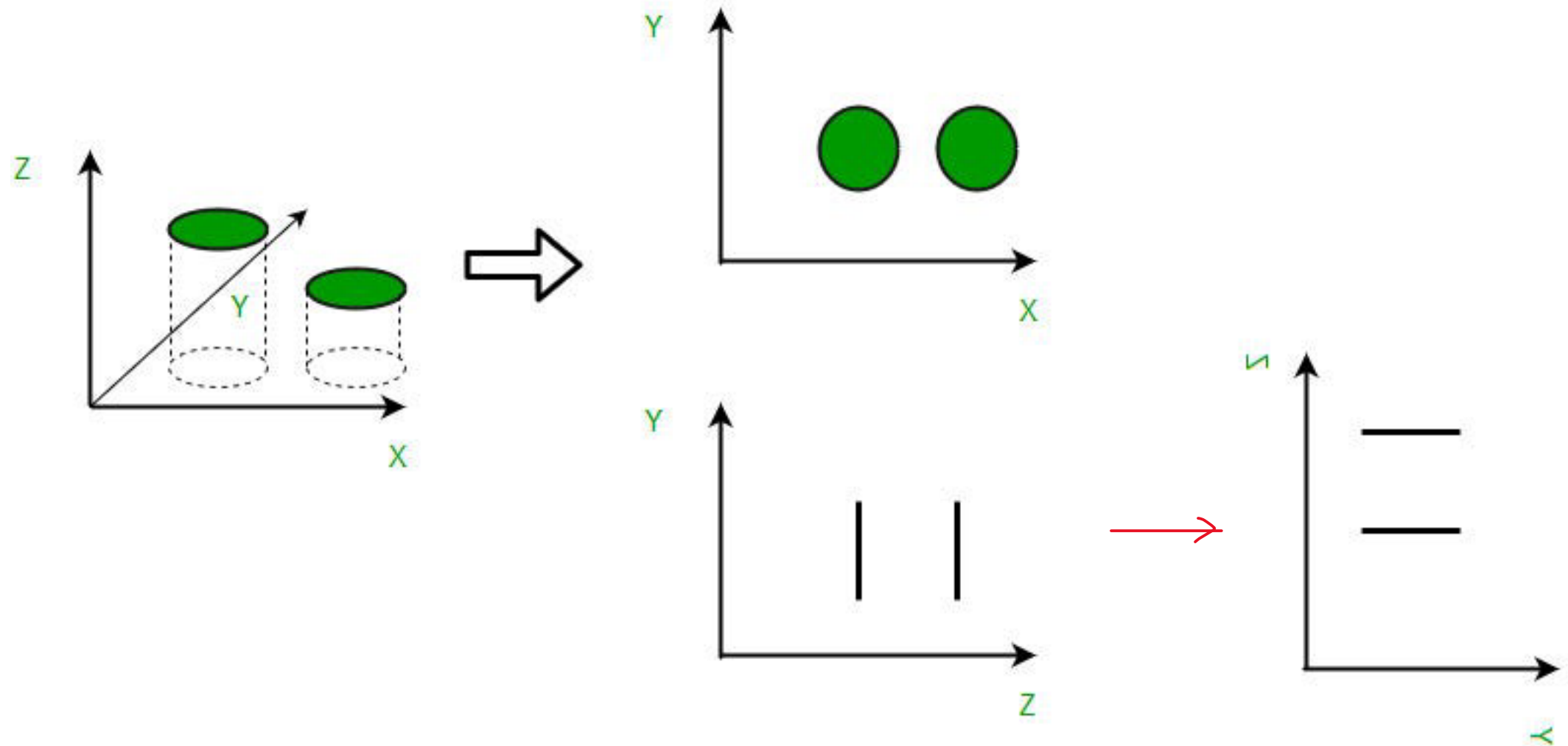
Process of identifying and selecting the most relevant features (variables, predictors) in a dataset for use in a ML model.

- Univariate Selection: Use statistical tests (e.g., ANOVA, chi-square) to select the best features.
- Model-Based Selection: Use feature importance scores from models like Random Forests or Gradient Boosting.
- Regularization: Methods like L1 regularization (Lasso) can automatically shrink irrelevant features to zero.
- Recursive Feature Elimination (RFE): Iteratively remove features and retrain the model to find optimal subsets.

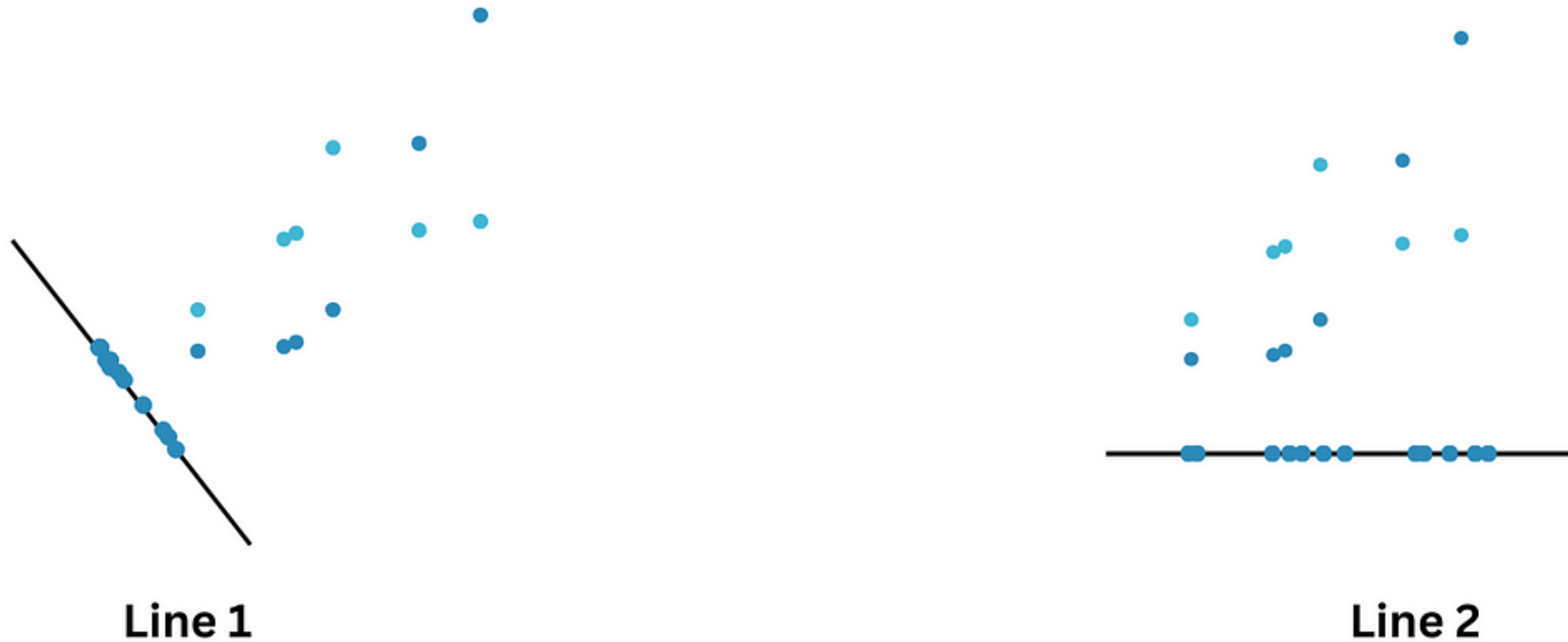
Feature Reduction (Dimensionality Reduction)

- Principal Component Analysis (PCA): Transform features into orthogonal components with maximal variance.
- Linear Discriminant Analysis (LDA): Reduce dimensions while maximizing class separability.
- t-SNE or UMAP: Visualize high-dimensional data in 2D/3D while preserving structure.

Dimensionality Reduction, $3 \rightarrow 2$

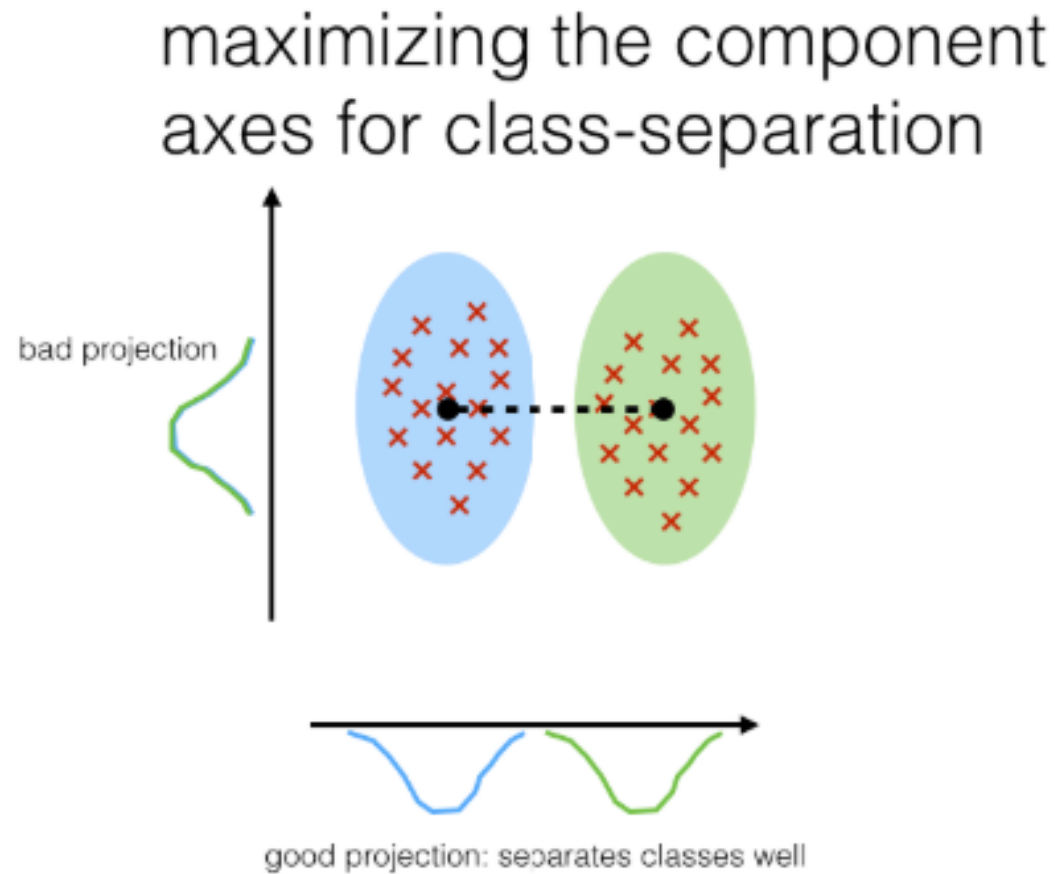


Dimensionality Reduction - Data points are scattered in a high dimensional space, the target is to *project* all those points on a line ensuring they are spaced out as much as possible.



The two lines represent projections of the same points, but the line 2 is optimized by maximizing the distance between the points, making it a more effective arrangement.

LDA (Linear Discriminant Analysis) idea



Feature Engineering (FE) Tasks

- Feature Creation
- Feature Transformation
- Feature Selection
- Feature Reduction (Dimensionality Reduction)
- Others Techniques
 - Handling Missing Values
 - Feature Encoding for Complex Data
 - Feature Noise Reduction
 - Sparse Data Handling
 - Embedding Features:

Handling Missing Values

- Imputation: Fill missing values with mean, median, mode, or using predictive models.
- Indicator Features: binary flags that denote whether a particular value in the dataset is missing. They are an additional **column** added to the dataset, where:
 - 1 (True) indicates a missing value.
 - 0 (False) indicates the presence of a value.

Feature Encoding for Complex Data

- Time Series: Generate lag, rolling averages, or trends from time-based data.
- Images: Use feature extractors like convolutional filters or embeddings from pretrained models.
- Text: Use TF-IDF, word2vec, or transformer-based embeddings like BERT.

Feature Noise Reduction

- Outlier Handling: Remove, cap, or transform extreme values.
 - "capping" refers to limiting the values of a feature to a specified maximum (upper cap) and/or minimum (lower cap).
- Filtering Noise: Use smoothing techniques or signal-processing methods to reduce variability.

Sparse Data Handling

- Use techniques to represent large sparse matrices efficiently.

Embedding Features

- Convert categorical variables to dense embeddings for deep learning models.

Python FE Techniques

- <https://medium.com/kgxperience/feature-engineering-for-machine-learning-a-step-by-step-guide-part-1-33b520c67137>
- <https://towardsdatascience.com/feature-engineering-for-machine-learning-3a5e293a5114>

Feature Engineering for Machine Learning

- <https://towardsdatascience.com/feature-engineering-for-machine-learning-eb2e0cff7a30>
- ...

Review (Feature Engineering Tasks)

- *New* Feature Creation
- Feature Transformation
- Feature Selection
- Feature Reduction
- PCA
- Polynomial Features
- RFE
- Min-Max scaling

Review (Feature Engineering Tasks)

-
- The diagram consists of two columns of feature engineering tasks and techniques. Red lines connect the tasks to the techniques as follows: 'New Feature Creation' connects to 'Polynomial Features'; 'Feature Transformation' connects to 'PCA'; 'Feature Selection' connects to 'Recursive Feature Elimination (RFE)'; and 'Feature Reduction' connects to 'Min-Max scaling'.
- *New Feature Creation*
 - Feature Transformation
 - Feature Selection
 - Feature Reduction
 - PCA
 - Polynomial Features
 - Recursive Feature Elimination (RFE)
 - Min-Max scaling

FE using EC

- Feature engineering and evolutionary computation, when combined, can significantly enhance machine learning outcomes by automating feature discovery and optimization.
- **Use genetic algorithms to select subsets of features that maximize model accuracy.** (See more in next slide)
- Genetic programming can be used to construct new features that enhance predictive power.

Using genetic algorithms (GAs) for feature selection

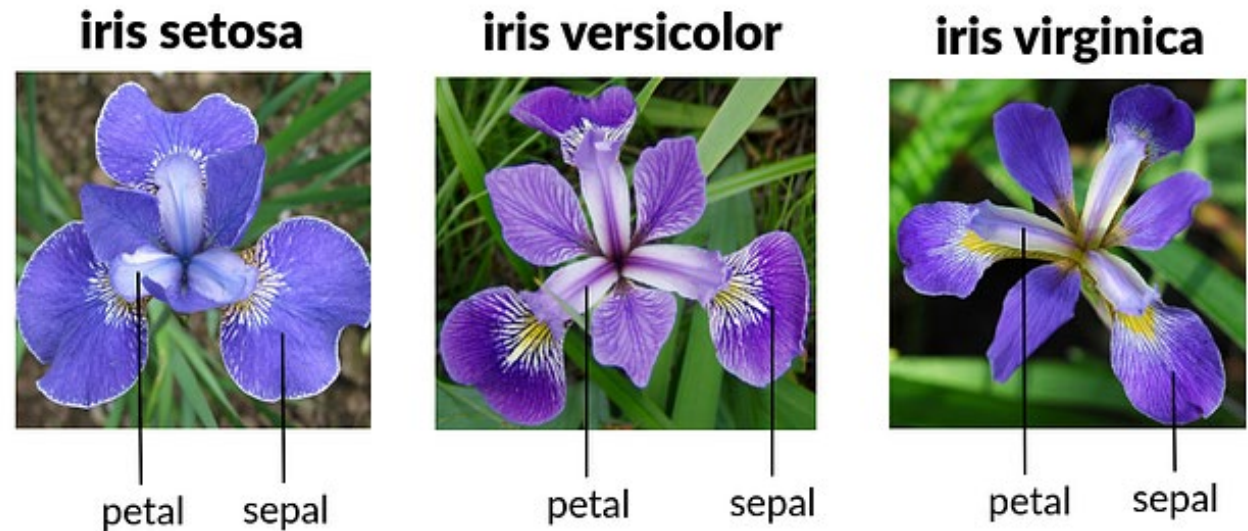
- An effective way to identify subsets of features that improve model accuracy while reducing computational complexity.
- It can
 - Improve model accuracy.
 - Reduce overfitting.
 - Decrease computational cost!

Why Evolutionary Computation?

- How many subsets of $\{1, 2, 3\}$?
 $\{ \{\}, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\} \}$ $2^3 = 8$ (the size of the power set)
- Total possible subsets of a set with n feature is 2^n
- Time complexity to evaluate every possible subset to find the global optima is $O(2^n)$, NP-hard problem
 - If $n=10$, 1,024. (Doable)
 - If $n=20$, ~1 million subsets (Slow, but possible)
 - If $n=50$, $\sim 1.12 \times 10^{15}$ subsets (Impossible)
- Methods?
 - Greedy Methods (e.g. RFE, Stepwise Selection) by math people: Fast, but they get stuck in "local optima" (they make short-term decisions).
 - Evolutionary Computation by CS people: Possibly slower than greedy, but much better at finding the global optimum.

Example: The Iris dataset

- Input Features (Input Variables): 4 features, measurements of the flowers. numeric and continuous:
 - Sepal length (cm)
 - Sepal width (cm)
 - Petal length (cm)
 - Petal width (cm)
- Target (Output Variable):
 - 0: Iris setosa
 - 1: Iris versicolor
 - 2: Iris virginica
- The dataset contains 150 samples, evenly distributed:
- 50 samples per class (species).



Sampe Python Code (Using DecisonTreeClassifier) on Canvas

Perfect problem for BGA. [0 1 1 1] means first feature not included.

■ **FeatureSelect_iris_GA.ipynb (*)** → [1 2 3] 96.6%

■ **FeatureSelect_iris_DEAP.ipynb** → [1 2 3] 96.0%

(*) Most cases it finds [0 1 2 3] with 96.6%

What did we find here?

- 0 • Sepal length (cm)
- 1 • Sepal width (cm)
- 2 • Petal length (cm)
- 3 • Petal width (cm)

